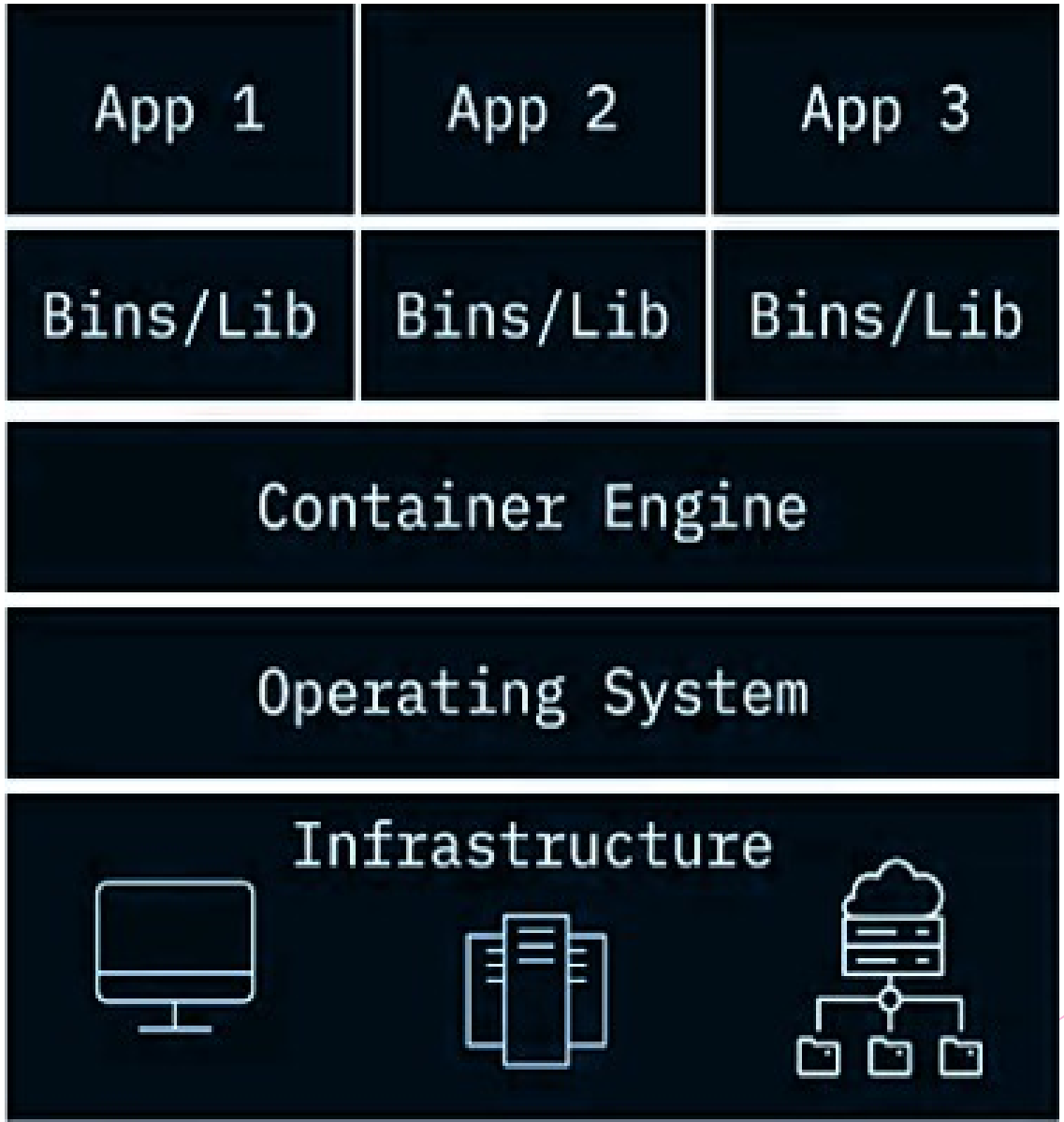
Module 3

Overview of containerization: introduction to docker containers, development of containerized application, benefits and overheads of containerization. Overview of virtualization, docker installation on multiple OS

Overview of containerization

- Cloud-native is the newest application development approach for building scalable, dynamic, hybrid cloud-friendly software.
- Container technology is a powerful part of that approach.
- Containers solve the problem of making software portable so that applications can run on multiple platforms.
- It enables applications to run reliably across different computing environments by packaging them with all their dependencies into isolated units called **containers**.

Containerized Architecture Layers



- **Infrastructure Layer**
  - Physical or virtual hardware (servers, CPUs, memory, storage, networking).
- **Operating System Layer**
  - Host OS (Linux, Windows, etc.) that manages hardware resources.
  - Containers share this kernel.
- **Container Engine Layer**
  - Software like Docker Engine, containerd, or CRI-O.
  - Manages container lifecycle, resource allocation, and isolation.

- **Application & Dependencies Layer**
  - Container images holding application code, libraries, runtimes, and configs.
  - Portable and reproducible across environments.

Why use containers?

1. **Consistency across environments**

Containers package the application **with all its dependencies** (libraries, runtimes, configs).This eliminates the classic *“works on my machine”* problem.

Whether you run it on a laptop, a server, or in the cloud, the behavior is identical.

2. **Portability**

Containers are platform-agnostic.

You can build once and run anywhere: on-premises, in the cloud, or hybrid setups.

This makes them ideal for multi-cloud and microservices architectures.

3. **Efficiency**
containers share the host OS kernel, so they are **lighter than virtual machines**.
They start in seconds, use fewer resources, and allow higher density on the same hardware.
4. **Scalability**
containers can be replicated and scaled easily using orchestration tools like **kubernetes** or **docker swarm**.
This makes them perfect for handling variable workloads and traffic spikes.

5. **Isolation & security**
containers isolate applications from each other using **namespaces** and **cgroups**.
This prevents conflicts between dependencies and improves security by limiting resource access.
6. **Devops & CI/CD integration**
containers fit seamlessly into **continuous integration/continuous deployment pipelines**.
They allow automated testing, deployment, and rollback with minimal effort.
7. **Microservices-friendly**
each microservice can run in its own container, independently deployable and scalable.
This aligns perfectly with modern cloud-native application design.

Who uses containerization? Real-life examples

- **Netflix:** Netflix uses containerization to make it easier for the company to scale its services and meet the demands of millions of users. Netflix designed a container management platform known as Titus to power streaming, content systems, and user recommendations.
- **Uber:** Uber leveraged containerization to manage its growing user base more efficiently. It did this by scaling Apache Hadoop with Docker containers.
- **Google:** Google was one of the early adopters of container technology. It even developed Kubernetes, an open-source container orchestration platform that has become an industry standard.

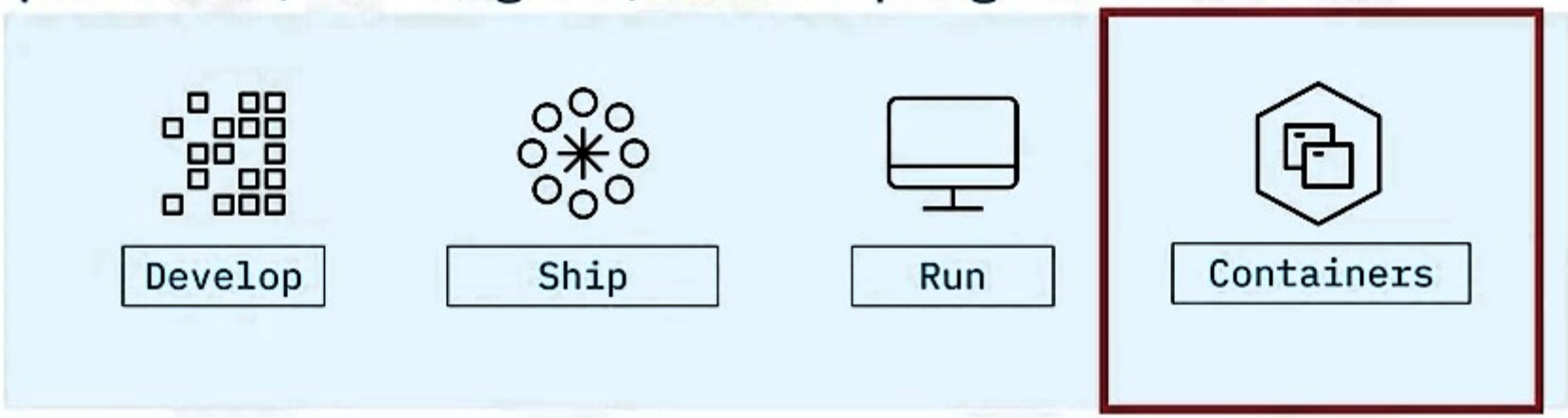
Container vendors

Docker	• Robust and most popular container platform today
Podman	• Daemon-less architecture providing more security than Docker containers
LXC	• Preferred for data-intensive apps and ops
Vagrant	• Offers highest levels of isolation on the running physical machine

- Organizations are moving to containers to overcome challenges around isolation, utilization, provisioning , performance, and more.
- A container is a standard unit of software that encapsulates everything needed to build, ship and run applications.
- Containers are operating system, programming language, and platform independent.
- They lower deployment time and costs, improve utilization, automate processes and support next gen applications (microservices).

Docker containers

Available since 2013, Docker is an open platform, or engine, where programmers can:



- Package Software into Standardized Units for Development, Shipment and Deployment
- A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.
- A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

- Container images become containers at runtime and in the case of Docker containers – images become containers when they run on Docker Engine.
- Available for both Linux and Windows-based applications, containerized software will always run the same, regardless of the infrastructure.
- Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging.

- Docker containers that run on Docker Engine:
  Standard: Docker created the industry standard for containers, so they could be portable anywhere
  Lightweight: Containers share the machine's OS system kernel and therefore do not require an OS per application, driving higher server efficiencies and reducing server and licensing costs
  Secure: Applications are safer in containers and Docker provides the strongest default isolation capabilities in the industry

Docker underlying technologies

- Docker relies on several **underlying technologies** from the Linux kernel and container ecosystem to provide lightweight, isolated, and efficient environments for running applications.

1. Namespaces (Process Isolation)
   - Namespaces are a Linux kernel feature that isolate system resources for each container. Some of them are..
     - **PID namespace**: Isolates process IDs so containers can't see each other's processes.
     - **NET namespace**: Provides separate network interfaces and IP addresses.
     - **User namespace**: Maps user and group IDs inside containers to host IDs.

2. Control Groups (cgroups)

- Control groups manage and limit resource usage for containers.
- Restrict CPU, memory, disk I/O, and network bandwidth.
- Prevent containers from consuming excessive resources.
- Enable prioritization and fair resource allocation.
- This is critical for multi-container environments and production workloads.

3. Union File Systems (UnionFS)

- UnionFS allows Docker to build images in layers.
  - Combines multiple file system layers into one unified view.
  - Each Docker image layer is stacked on top of the previous one.
  - Examples: AUFS, OverlayFS, Btrfs.
- This makes Docker images efficient, portable, and fast to build.

4. Container Runtime

- The container runtime is responsible for starting and managing containers.
- **runc**: A lightweight CLI tool that runs containers according to the OCI (Open Container Initiative)- open standard under the Linux Foundation that defines container image formats and runtime specifications.
- **containerd**: A daemon that manages container lifecycle (start, stop, delete, etc.).
- Docker uses containerd and runc under the hood to execute containers.

5. Docker Engine

- The high-level component that ties everything together.
  - Docker Daemon (dockerd): Manages containers, images, volumes, and networks.
  - Docker CLI (docker): Sends commands to the daemon.
  - REST API: Enables programmatic access to Docker features.

6. Networking Stack

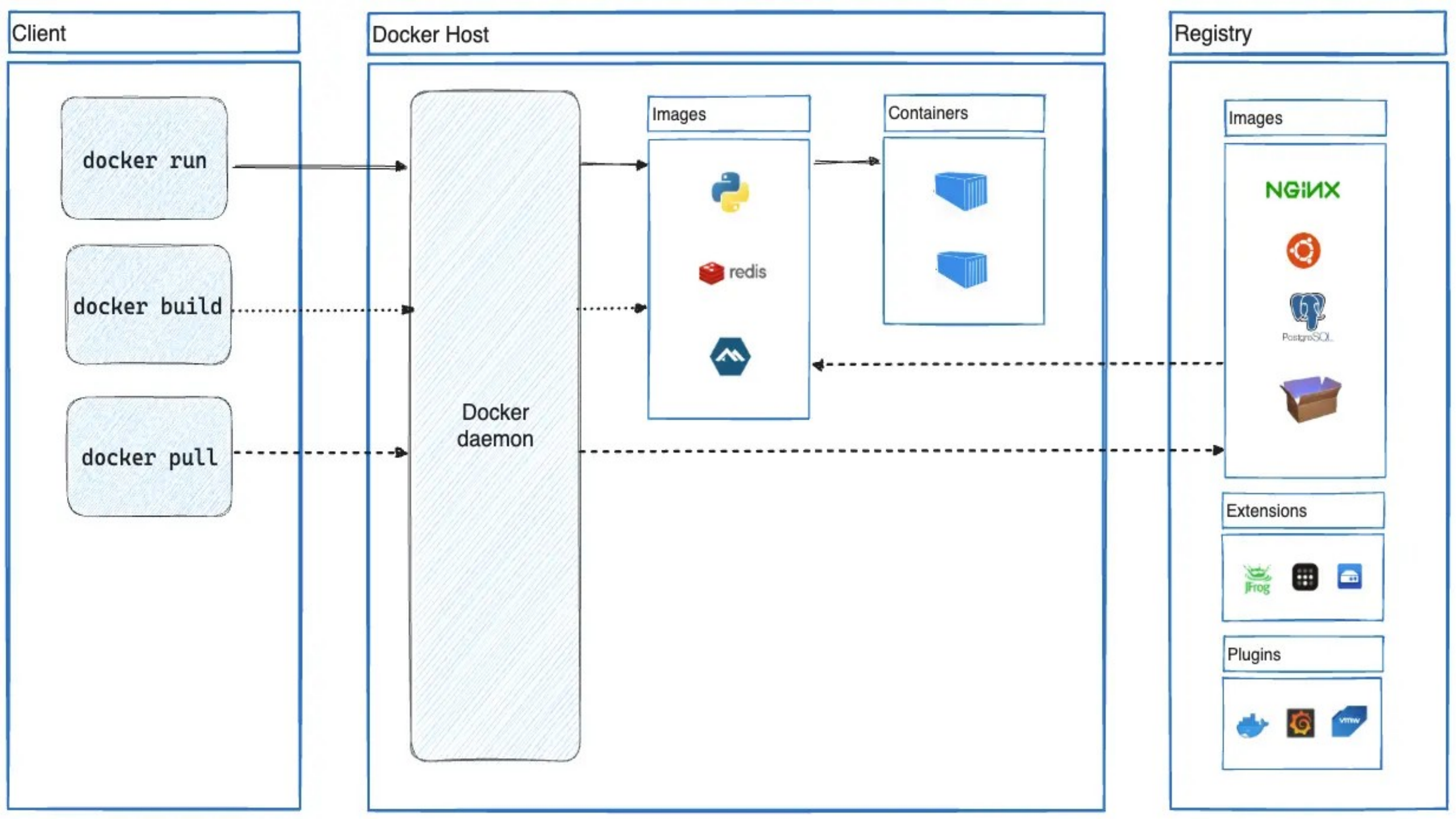
- Docker uses virtual networking to connect containers.
  - **Bridge network**: Default network for containers on a single host.
  - **Overlay network**: Connects containers across multiple hosts (used in Swarm).
  - **Host network**: Shares the host's network stack.
  - **Macvlan**: Assigns MAC addresses for direct access to physical networks.

7. Storage Drivers

- Docker uses storage drivers to manage container filesystems.
- Examples: Overlay2, Device Mapper.
- These drivers interact with UnionFS to provide layered storage.

Docker Architecture

- Docker uses a client-server architecture.
- The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers.
- The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon.
- The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.


Client
Docker Host
Registry
docker run
docker build
docker pull
Docker daemon
Images
Containers
redis
Images
NGINX
PostgreSQL
Extensions
JFrog
Plugins

1. The Docker client

- The Docker client (**docker**) is the primary way that many Docker users interact with Docker.
- When you use commands such as **docker run**, the client sends these commands to **dockerd**, which carries them out.
- Docker build is used to **create a Docker image** from a Dockerfile.
- Docker pull used to **download an image** from Docker Hub or a registry.
- The **docker** command uses the Docker API. The Docker client can communicate with more than one daemon.

2. Docker Host

- The physical or virtual machine where the Docker Daemon runs.
- Provides CPU, memory, storage, and networking resources for containers.
- Containers share the host OS kernel but remain isolated via namespaces and cgroups.

- ✓ *Docker Daemon (dockerd)*
  - The background service that manages Docker objects.
  - Responsibilities:
    - Builds images from Dockerfiles.
    - Runs and manages containers.
    - Handles networking and volumes.
    - Interacts with registries to pull/push images.

- *Container runtime*
  - A software component that creates and manages containers according to the OCI (Open Container Initiative) specifications.
  - It handles the actual execution of container processes, isolation, and resource allocation.
  - Docker relies on container runtimes under the hood to make containers work.

- Key Container Runtimes in Docker

1. **containerd**

- A daemon that manages the **lifecycle of containers**:
  - Pulling images from registries
  - Creating, starting, stopping, and deleting containers
  - Managing storage and networking for containers
- Acts as the **high-level runtime** in Docker.
- Provides APIs for Docker Daemon to interact with containers.

2. **runc**

- A lightweight CLI tool that **creates and runs containers** based on OCI runtime specifications.
- Handles low-level tasks like:
  - Setting up namespaces (process, network, filesystem isolation)
  - Applying cgroups (resource limits for CPU, memory, I/O)
  - Executing the container process
- Acts as the **low-level runtime** in Docker.

3. **OCI (Open Container Initiative)**

- A standard specification for container runtimes and images.
- Ensures compatibility across different container technologies (Docker, Podman, CRI-O).

- ✓ *Docker Objects*
- **Images**: Immutable templates built from Dockerfiles.
  Immutable, read-only templates used to create containers.
  - Built from Dockerfiles and stored in registries (Docker Hub, GitHub Container Registry, private registries).
  - Layered structure makes them efficient and reusable.
  - Example: nginx is an image that can be pulled and run as a container.

- **Containers**: Running instances of images.
  - Lightweight, isolated environments with their own filesystem, processes, and network.
  - Ephemeral by default (deleted when stopped), but can be configured with volumes for persistence.
  - Example: Running creates a container from the Nginx image.

- **Volumes**: Persistent storage outside container's ephemeral filesystem.
  - Persistent storage mechanism for containers.
  - Stores data outside the container's ephemeral filesystem.
  - Useful for databases, logs, and shared files.
  - Example: creates a named volume.

- **Networks**: Enable communication between containers and external systems.
  - Provide communication between containers and external systems.
  - Types of Docker networks:
    - Bridge → Default, connects containers on a single host.
    - Host → Shares host's network stack.
    - Overlay → Connects containers across multiple hosts (used in Swarm/Kubernetes).
    - Macvlan → Assigns MAC addresses for direct network access.

- **Plugins:** modular add-ons that extend Docker Engine.
  - They allow Docker to integrate with external systems or provide features too specialized to ship with the default installation.
  - Once installed, plugins behave like native Docker components — you reference them in Docker commands without interacting with them directly.
  - Some are Logging plugins, Authorized plugins, Volume plugins, Network plugins.

- **Dockerfile:** is the blueprint for building Docker images.
  - It’s a simple text file containing a series of instructions that Docker executes sequentially to assemble an image.
  - Automates the process of creating Docker images.
  - Ensures consistency across environments (development, staging, production).
  - Acts like a recipe: defines base image, dependencies, configuration, and startup commands.

- Structure of a Dockerfile:
- Each line is an instruction followed by arguments. Common instructions include:
  - **FROM** → Base image (e.g., `FROM ubuntu:20.04`).
  - **RUN** → Execute commands during build (install packages, configure system).
  - **COPY / ADD** → Copy files into the image.
  - **WORKDIR** → Set working directory inside the container.
  - **CMD** → Default command when container starts.

3. Docker Registry

- A repository system that stores Docker images.
- Acts as a server-side application that stores Docker images.
- Provides repositories, each containing multiple versions (tags) of an image.
- Supports push (upload) and pull (download) operations.
- Enables collaboration and deployment across environments.

- Key Components of a Registry
  - Repositories: Collections of related images (e.g., repository contains multiple versions).
  - Tags: Labels that identify specific versions (,).
  - Authentication & Authorization: Controls who can push/pull images.
  - API: Allows automation tools (CI/CD pipelines) to interact with the registry

- Types of Registries
  1. Public Registries
     - Docker Hub: The default public registry, widely used for open-source images.
     - GitHub Container Registry: Integrated with GitHub for storing images alongside code.
     - Google Artifact Registry / AWS ECR / Azure Container Registry: Cloud provider registries.

2. Private Registries
   - Used by enterprises for internal images.
   - Provides access control, security scanning, and compliance.
   - Example: Harbor, JFrog Artifactory.s teams to share and distribute containerized applications.

Workflow in action

- Developer writes a **Dockerfile** with app dependencies.
- **Docker Client** sends a build command to the **Docker Daemon**.
- Daemon uses **containerd/runc** to build and run containers.
- Images are stored in a **registry** for reuse and distribution.
- Containers run on the **Docker Host**, isolated but sharing the OS kernel.
- Networking and volumes connect containers and persist data.
- Monitoring tools integrate via the REST API for visibility.

Development of containerized application

- Containers encapsulate an application as a single executable package of software that bundles application code together with all of the related configuration files, libraries, and dependencies required for it to run.
- Containerized applications are “isolated”.
- An open source runtime engine (such as the Docker runtime engine) is installed on the host’s operating system and becomes the channel for containers to share an operating system with other containers on the same computing system.
- Other container layers, like common bins and libraries, can also be shared among multiple containers.

- A containerized app typically has a much smaller footprint than a virtual machine configured to run the same app.
- This smaller footprint is because a virtual machine has to supply the entire operating system and associated supporting environment.
- A Docker container doesn't have this overhead because Docker uses the operating system kernel of the host computer to power the container.
- Downloading and starting a Docker image is faster and more space-efficient than downloading and running a virtual machine that provides similar functionality.

- Create a containerized app by building an image that contains a set of files and a section of configuration information Docker uses.
- Run the app by asking Docker to start a container based on the image.
- When the container starts, Docker uses the image configuration to determine what application to run inside the container.
- Docker provides the operating system resources and the necessary security.
- It ensures that containers are running concurrently and remain relatively isolated.

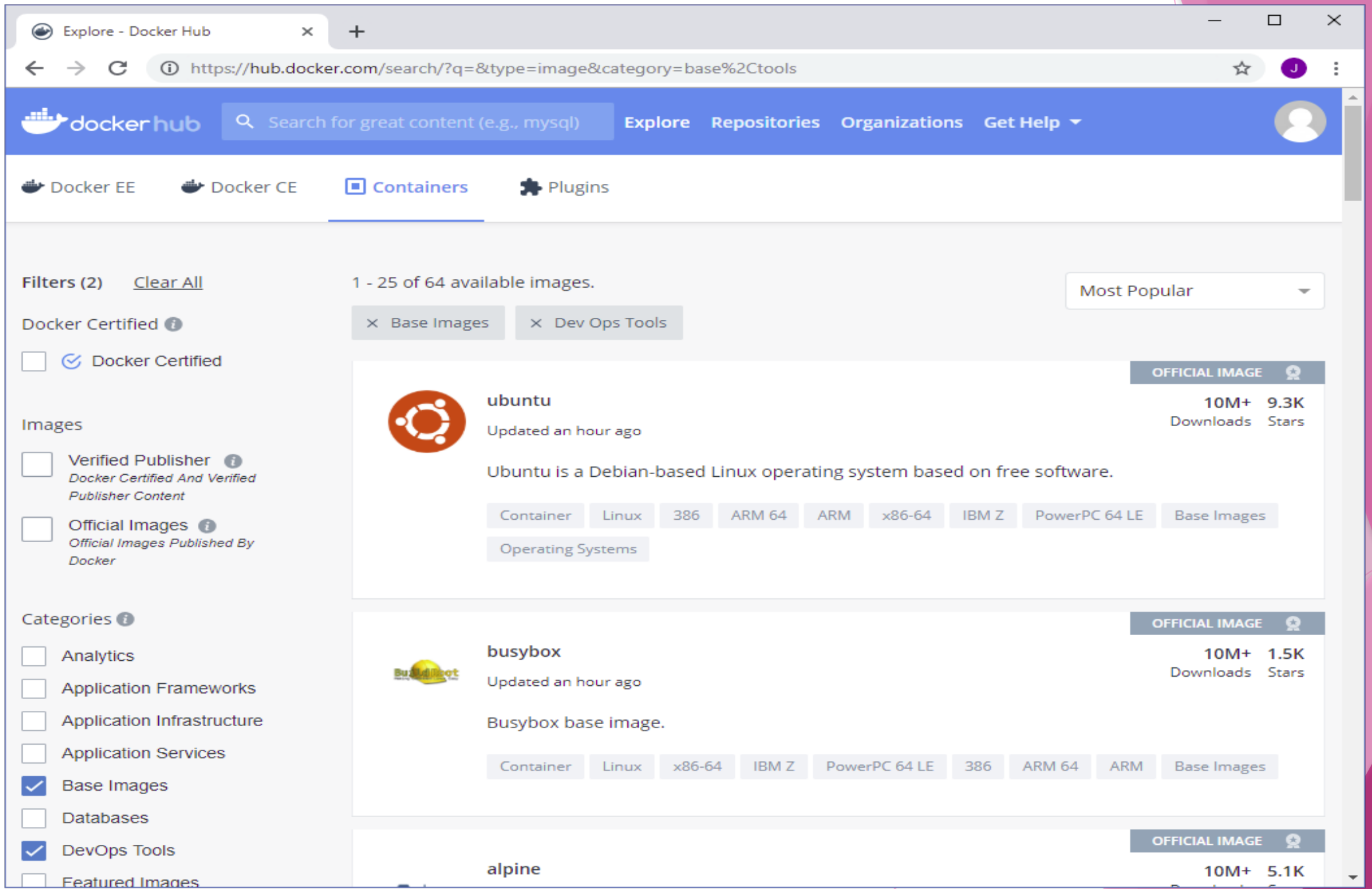
- For production systems, Docker is available for server environments, including many variants of Linux and Microsoft Windows Server 2016.
- Many vendors also support Docker in the cloud.
- For example, you can store Docker images in Azure Container Registry and run containers with Azure Container Instances.
- Docker images are stored and made available in registries.
- A registry is a web service to which Docker can connect to upload and download container images.
- The most well-known registry is Docker Hub, which is a public registry.

- A **registry** is organized as a series of repositories.
- Each **repository** contains multiple Docker images that share a common name and generally the same purpose and functionality.
- These images normally have different versions, identified with a tag.
- A **repository** is also the unit of privacy for an image.
- If you don't wish to share an image, you can make the repository private. You can grant access to other users with whom you want to share the image.

Browse Docker Hub and pull an image

- Docker Hub contains many thousands of images.
- You can search and browse a registry using Docker from the command line or the Docker Hub website.
- The website allows you to search, filter, and select images by type and publisher.

- The figure below shows an example of the search page.



- You use the docker pull command with the image name to retrieve an image.
- By default, Docker will download the image tagged latest from that repository on Docker Hub if you specify only the repository name.
- ***docker pull mcr.microsoft.com/dotnet/samples:aspnetapp***

This example fetches the image with the tag ***aspnetapp*** from the ***mcr.microsoft.com/dotnet/samples:aspnetapp*** repository. This image contains a simple ASP.NET Core web app

- When you fetch an image, Docker stores it locally and makes it available for running containers.
- You can view the images in your local registry with the ***docker image list*** command

- The output looks like the following example:

```
REPOSITORY TAG IMAGE ID CREATED SIZE
mcr.microsoft.com/dotnet/samples   aspnetapp          6e2737d83726        6 days ago          263MB
```

- You can use the image name ID to reference the image in many other Docker commands.
- Use the docker run command to start a container. Specify the image to run with its name or ID.
- If you haven't docker pulled the image already, Docker will do it for you.
- ***docker run mcr.microsoft.com/dotnet/samples:aspnetapp***

- In this example, the command responds with the following message:

Console

warn:
Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[35]

No XML encryptor configured. Key {d8e1e1ea-126a-4383-add9-d9ab0b56520d} may be persisted to storage in unencrypted form.

Hosting environment: Production

Content root path: /app

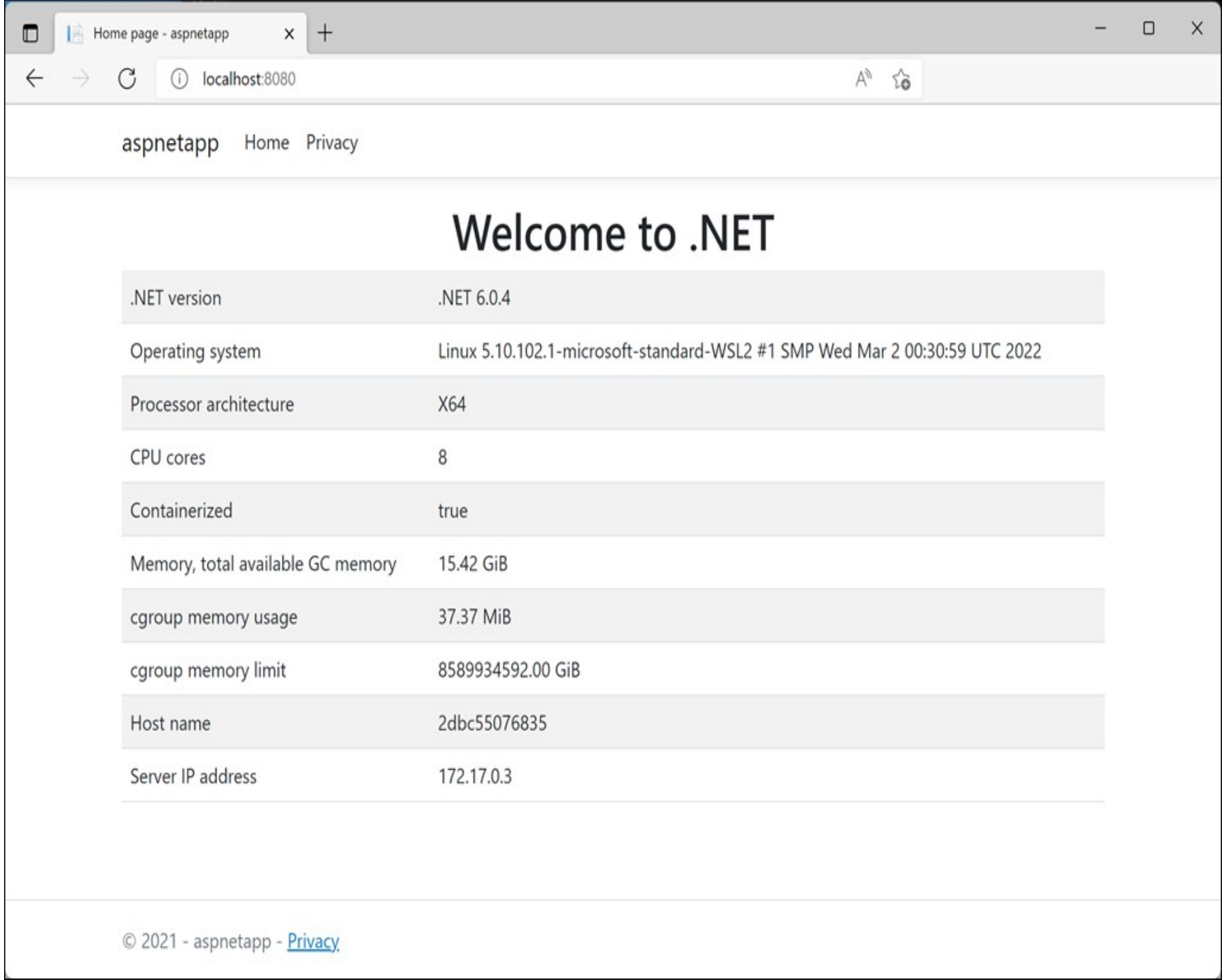
Now listening on: http://[::]:80

Application started. Press Ctrl+C to shut down.

- This image contains a web app, so it's now listening for requests to arrive on HTTP port 80. However, if you open a web browser and navigate to http://localhost:80, you won't see the app.
- By default, Docker doesn't allow inbound network requests to reach your container.
- You need to tell Docker to assign a specific port number from your computer to a specific port number in the container by adding the -p option to docker run.
- This instruction enables network requests to the container on the specified port.

- Additionally, the web app in this image isn't meant to be used interactively from the command line.
- When we start it, we want Docker to start it in the background and just let it run.
- Use the -d flag to instruct Docker to start the web app in the background.
- Press Ctrl+C to stop the image and then restart it as shown by the following example:
- ***docker run -p 8080:80 -d mcr.microsoft.com/dotnet/samples:aspnetapp***

- ***docker run -p 8080:80 -d mcr.microsoft.com/dotnet/samples:aspnetapp***
- The command maps port 80 in the container to port 8080 on your computer. If you visit the page http://localhost:8080 in a browser, you'll see the sample web app.



- If a running container makes changes to the files in its image, those **changes only exist in the container where the changes are made**.
- Unless you take specific steps to preserve the state of a container, these **changes are lost when the container is removed.**
- Similarly, **multiple containers based on the same image** that run simultaneously **don't share the files in the image.**
- **Each container** has its own **independent copy**. Any data written by one container to its filesystem isn't visible to the other.
- It's **possible to add writable volumes to a container.**
- A volume represents a filesystem that can be mounted by the container, and is made available to the application running in the container.
- The data in a volume does persist when the container stops, and multiple containers can share the same volume.

- View active containers with the docker ps command : ***docker ps***
- Stop an active container with the docker stop command, specifying the container ID: ***docker stop cname***
- Restart a stopped container with the docker start command : ***docker start cname***
- Force a container to be stopped and removed with the -f flag to the docker rm command: ***docker container rm -f cname***
- remove an image from the local computer with the docker image rm command. Specify the image ID of the image to remove:
- ***docker image rm mcr.microsoft.com/dotnet/core/samples:aspnetapp***
- Containers running the image must be terminated before the image can be removed

Benefits and Overheads of Containerization

Benefits of Containerization

- **Portability**: Applications run consistently across different environments (development, testing, production, cloud, on-premises).
- **Scalability**: Containers can be quickly replicated or scaled up/down to meet demand.
- **Resource Efficiency**: Containers share the host OS kernel, making them lighter than virtual machines.

- **Faster Deployment**: Applications can be packaged and deployed rapidly, supporting CI/CD pipelines.
- **Isolation**: Each container runs independently, reducing conflicts between applications.
- **Microservices Support**: Encourages modular design, allowing teams to build, deploy, and scale services independently.
- **Improved Testing**: Developers can replicate production-like environments easily for testing.
- **Resilience**: Containers can be restarted or replaced quickly if they fail.

Overheads of Containerization

- **Complex Orchestration**: Managing large numbers of containers requires tools like Kubernetes, which add complexity.
- **Security Risks**: Containers share the host OS kernel, so vulnerabilities can spread if not properly isolated.
- **Monitoring Challenges**: Traditional monitoring tools may not work well with dynamic, short-lived containers.
- **Networking Overhead**: Container networking can be complex and introduce latency.

- **Storage Management**: Persistent storage across containers is harder to manage compared to traditional deployments.
- **Learning Curve**: Teams need to learn new tools and practices, which can slow adoption initially.
- **Resource Contention**: Poorly configured containers can compete for CPU/memory, affecting performance.

Overview of virtualization

- Virtualization is technology that you can use to create virtual representations of servers, storage, networks, and other physical machines.
- Virtual software mimics the functions of physical hardware to run multiple virtual machines simultaneously on a single physical machine.
- Businesses use virtualization to use their hardware resources efficiently and get greater returns from their investment.
- It also powers cloud computing services that help organizations manage infrastructure more efficiently.

- By using virtualization, you can interact with any hardware resource with greater flexibility.
- Physical servers consume electricity, take up storage space, and need maintenance.
- You are often limited by physical proximity and network design if you want to access them.
- Virtualization removes all these limitations by abstracting physical hardware functionality into software.
- You can manage, maintain, and use your hardware infrastructure like an application on the web.

- **Virtualization example**
  - Consider a company that needs servers for three functions:
    - Store business email securely
    - Run a customer-facing application
    - Run internal business applications
  - Each of these functions has different configuration requirements:
  - The email application requires more storage capacity and a Windows operating system.
  - The customer-facing application requires a Linux operating system and high processing power to handle large volumes of website traffic.

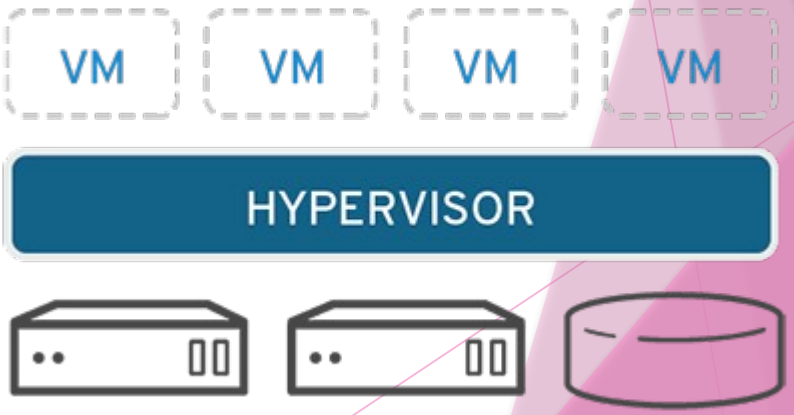
- The internal business application requires iOS and more internal memory (RAM).
- To meet these requirements, the company sets up three different dedicated physical servers for each application.
- The company must make a high initial investment and perform ongoing maintenance and upgrades for one machine at a time.
- The company also cannot optimize its computing capacity. It pays 100% of the servers' maintenance costs but uses only a fraction of their storage and processing capacities.

- ***Efficient hardware use***
  - With virtualization, the company creates three digital servers, or virtual machines, on a single physical server.
  - It specifies the operating system requirements for the virtual machines and can use them like the physical servers. However, the company now has less hardware and fewer related expenses.

- ***Infrastructure as a service***
  - The company can go one step further and use a cloud instance or virtual machine from a cloud computing provider such as AWS.
  - AWS manages all the underlying hardware, and the company can request server resources with varying configurations.
  - All the applications run on these virtual servers without the users noticing any difference.
  - Server management also becomes easier for the company’s IT team.

The main components of virtualization

- Virtualization relies on several key components to create and manage virtual environments.
- Each plays a vital role in ensuring the effective allocation of resources so multiple VMs can run simultaneously without interference.
  - Physical machine (server/computer)
  - Virtual machine (VMs)
  - Hypervisor



Physical machine (server/computer)

- The physical machine, also referred to as the “host machine” is the hardware (e.g., server or computer) that provides CPU, memory, storage and network resources for the virtual machines.

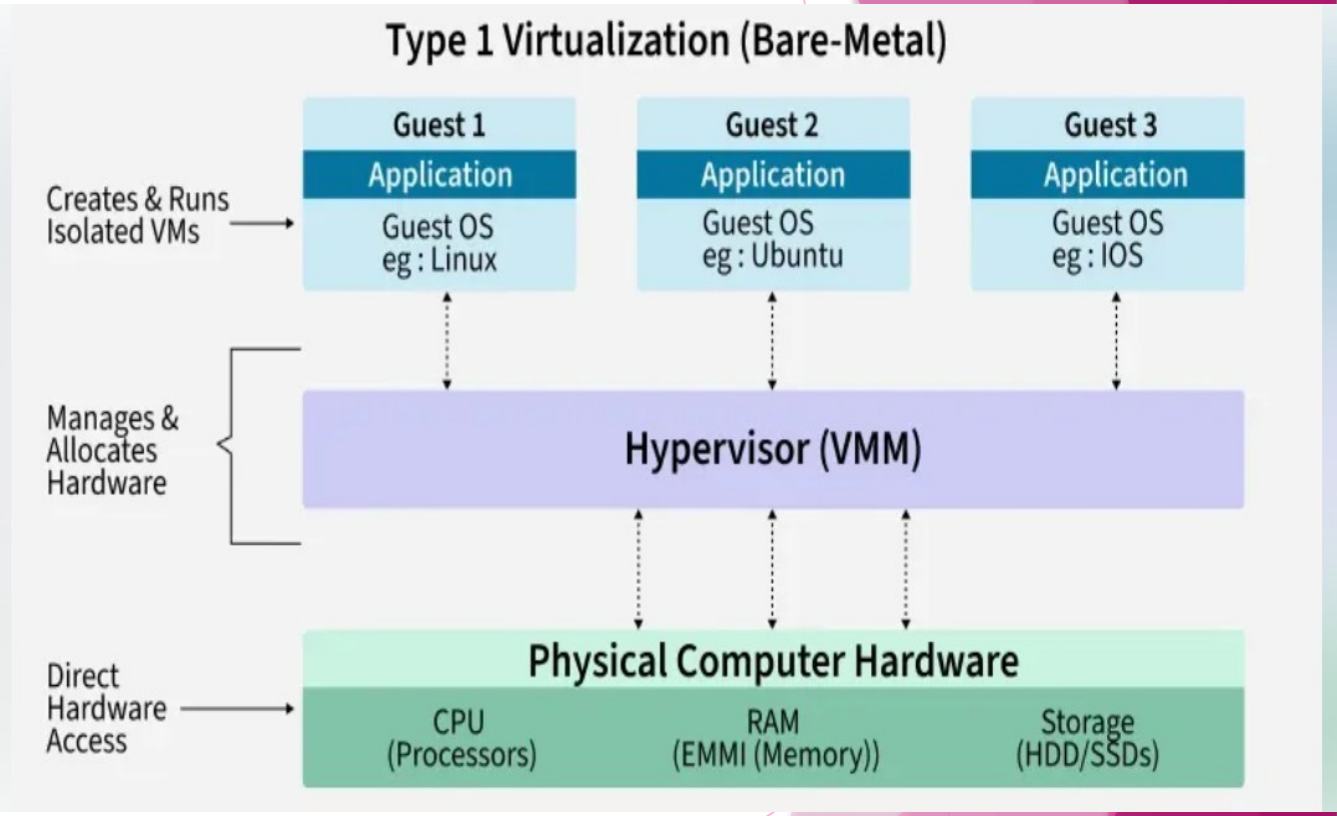
Virtual Machines (Cloud Instances)

- After installing virtualization software, you can create one or more virtual machines on your computer.
- Virtual machines (VMs) behave like regular applications on your system.
- The real physical computer is called the **Host**, while the virtual machines are called **Guests**.
- A single host can run multiple guest virtual machines.
- Each guest can have its own operating system, which may be the same or different from the host OS.
- Every virtual machine functions like a standalone computer, with its own settings, programs, and configuration.
- VMs access system resources such as **CPU, RAM, and storage**, but they work as if they are using their own hardware.

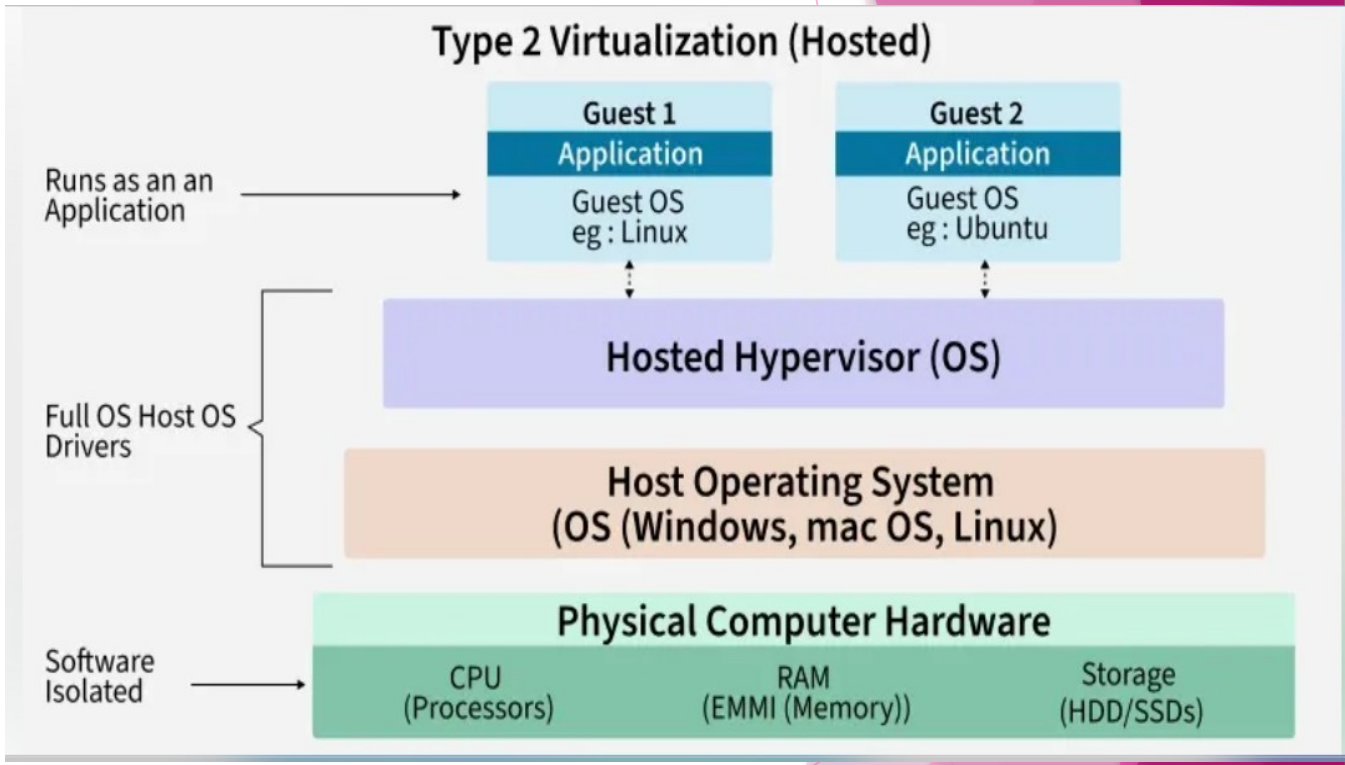
Hypervisors

- A hypervisor is the software layer that coordinates VMs.
- It serves as an interface between the VM and the underlying physical hardware, ensuring that each has access to the physical resources it needs to execute.
- It also makes sure that the VMs don't interfere with each other by impinging on each other's memory space or compute cycles.

- There are two types of hypervisors:
- **Type 1 hypervisors:**
- Type 1 or “bare-metal” hypervisors interact with the underlying physical resources, replacing the traditional operating system altogether.
- They most commonly appear in virtual server scenarios in which a software-based server is created by partitioning a physical server into smaller, self-contained segments, each capable of running its own operating system and applications.



- **Type 2 hypervisors:**
- Type 2 hypervisors run as an application on an existing OS.
- Most commonly used on endpoint devices to run guest operating systems, they carry a performance overhead because they must use the host OS to access and coordinate the underlying hardware resources.

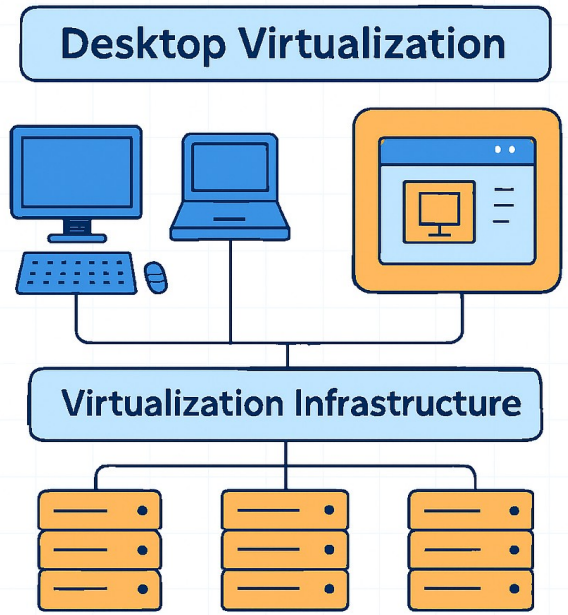


Types of virtualization

- Desktop virtualization
- Network virtualization
- Storage virtualization
- Data virtualization
- Application virtualization
- Data center virtualization
- CPU virtualization
- GPU virtualization
- Linux virtualization
- Cloud virtualization

Desktop Vitualization

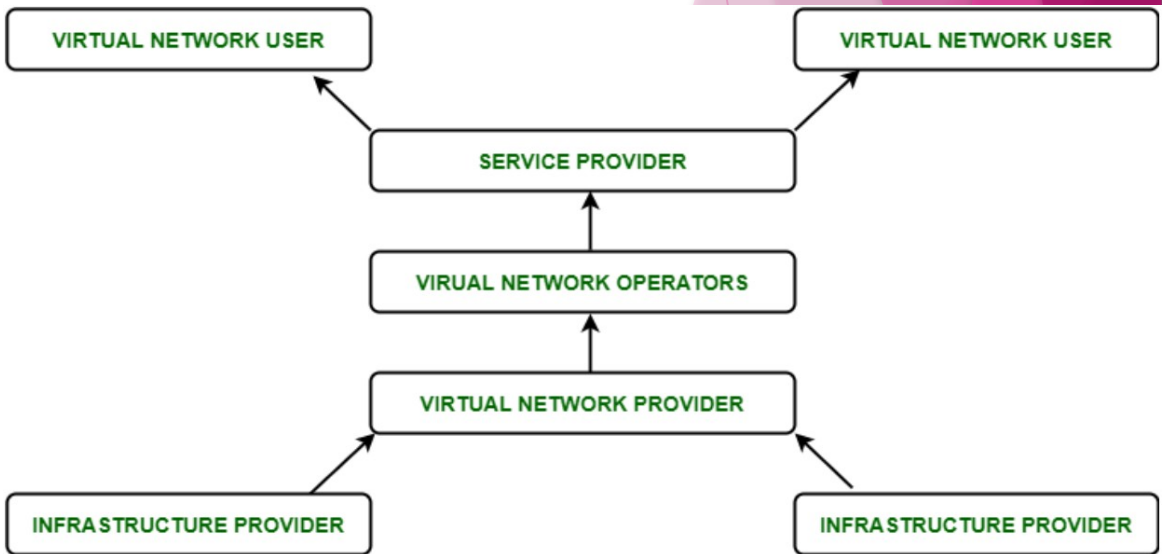
- Hosting a user's desktop environment on a centralized server. The user connects via a "thin client" (a basic PC).
  - Virtual desktop infrastructure (VDI)
    - In VDI deployment model, the operating system runs on a virtual machine (VM) hosted on a server in a data center.
    - The desktop image travels over the network to the end user’s device, where the end user can interact with the desktop (and the underlying applications and operating system) as if they were local.



- Remote desktop services (RDS)
  - Users remotely access desktops and Windows applications through the Microsoft Windows Server operating system.
  - Applications and desktop images are served via Microsoft Remote Desktop Protocol. From the end user's perspective, RDS and VDI are identical.
  - But because one instance of Windows Server can support as many simultaneous users as the server hardware can handle, RDS can be a more cost-effective desktop virtualization option.
- Desktop-as-a-service (DaaS)
  - In DaaS, VMs are hosted on a cloud-based backend by a third-party provider.
  - DaaS is readily scalable, can be more flexible than on-premise solutions, and generally deploys faster than many other desktop virtualization options.

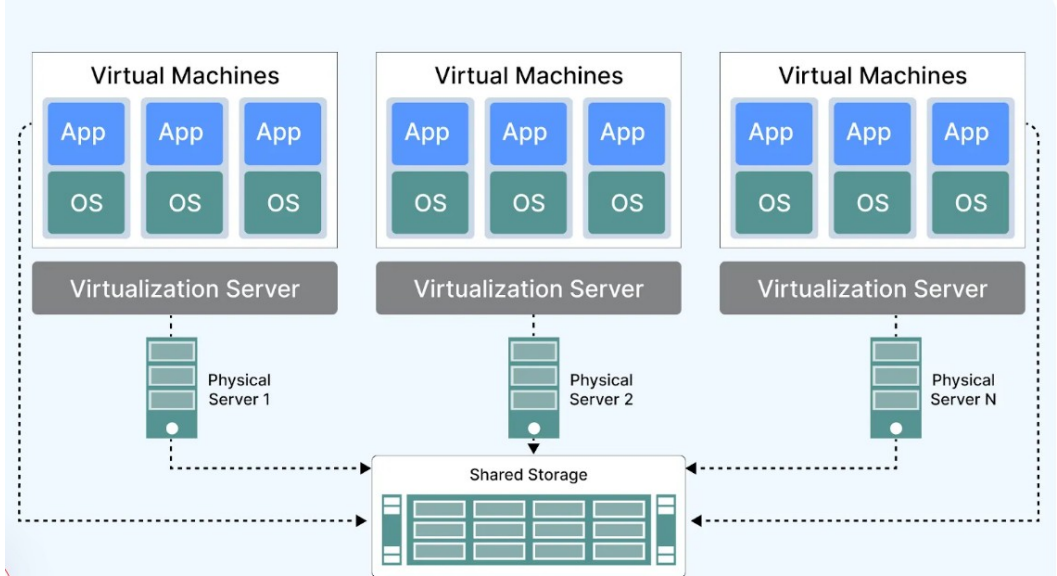
Network virtualization

- Network Virtualization is a process of logically grouping physical networks and making them operate as single or multiple independent networks called Virtual Networks.
- uses software to create a “view” of the network that an administrator can use to manage the network from a single console.
- It abstracts hardware elements and functions (e.g., connections, switches, routers, etc.) and abstracts them into software running on a hypervisor.
- The network administrator can modify and control these elements without touching the underlying physical components, which dramatically simplifies network management.



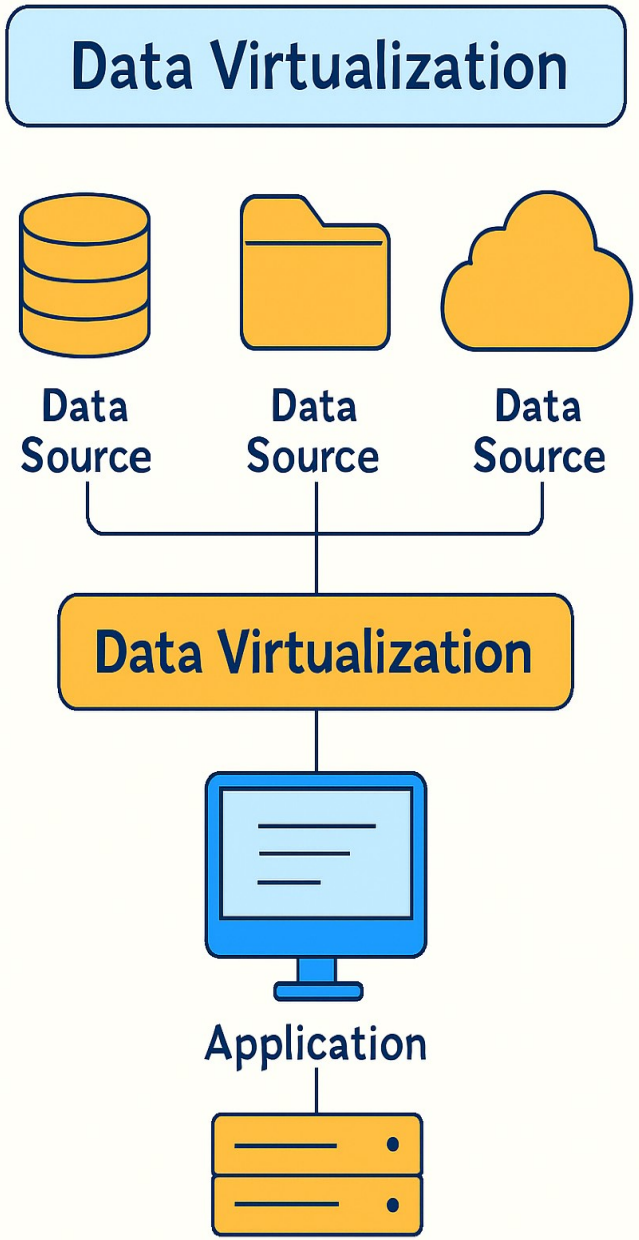
Storage Virtualization

- Enables all the storage devices on the network— whether they're installed on individual servers or standalone storage units—to be accessed and managed as a single storage device.
- You can pool the storage hardware in your data center, even if it is from different vendors or of different types.
- Storage virtualization uses all your physical data storage and creates a large unit of virtual storage that you can assign and control by using management software.



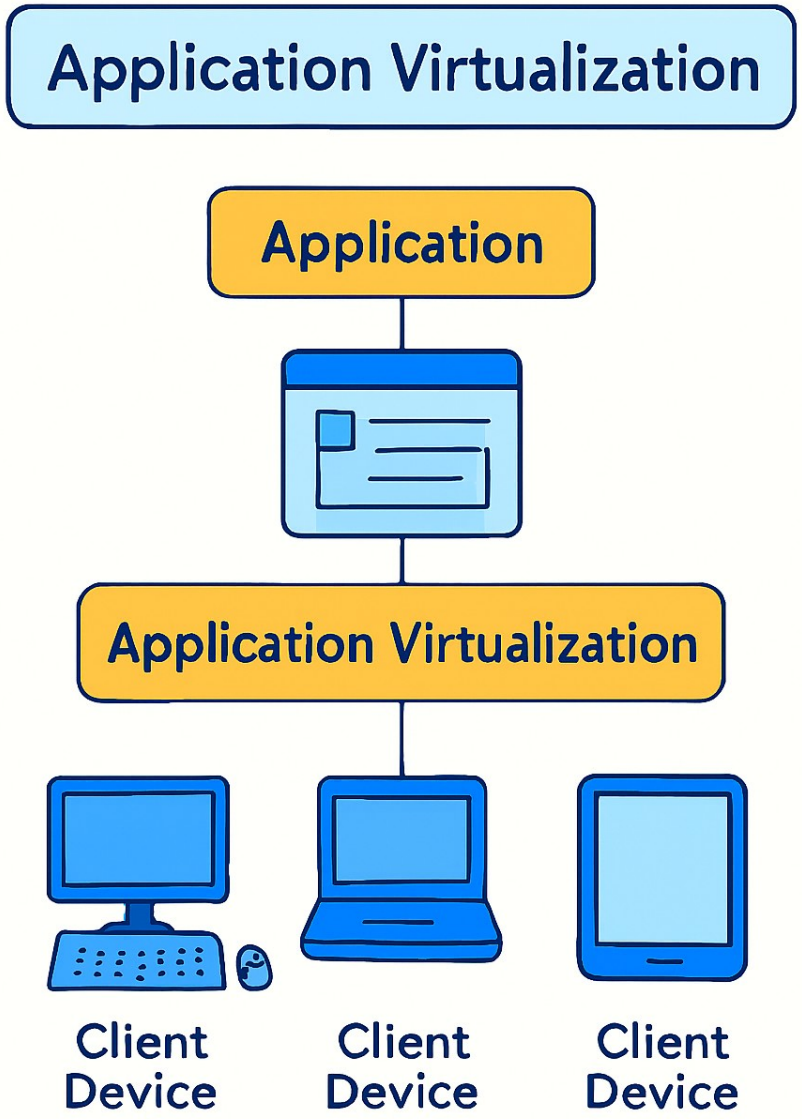
Data Virtualization

- An abstract layer that allows you to access data from multiple different sources (databases, files, cloud) as if it were in a single place, without moving the data.
- Modern enterprises store data from multiple applications, using multiple file formats, in multiple locations, ranging from the cloud to on-premise hardware and software systems.



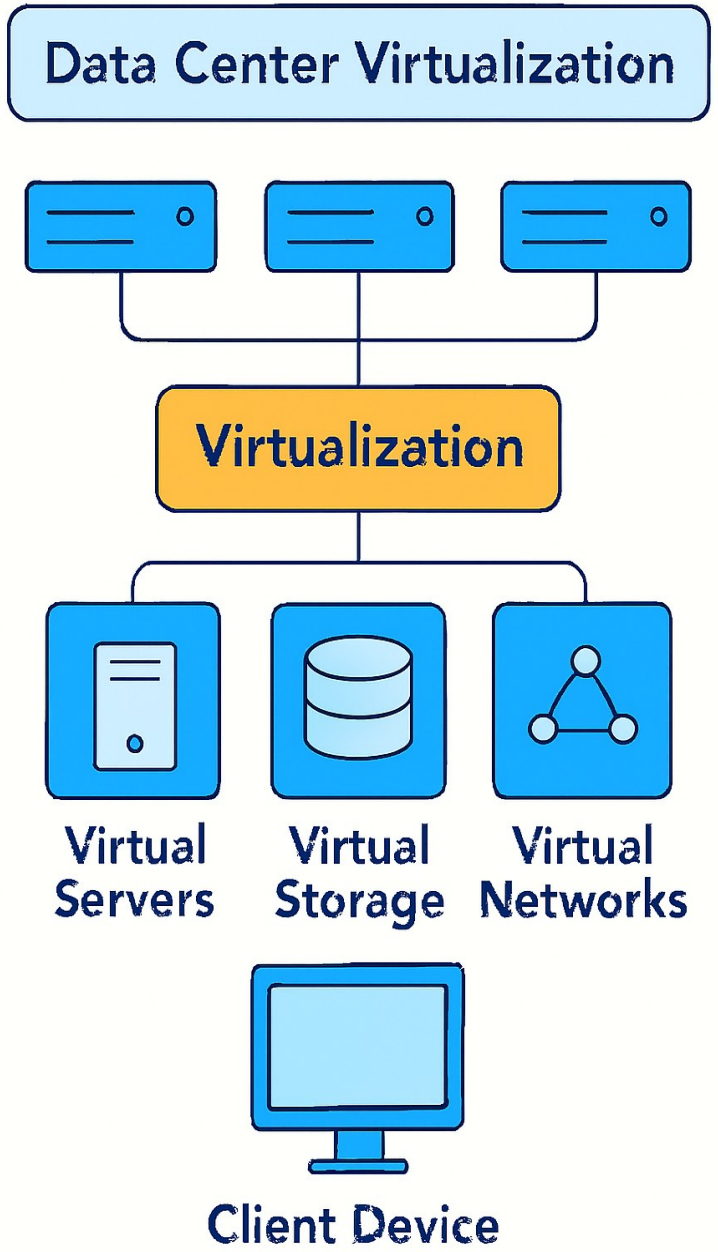
Application Virtualization

- Encapsulating an application so it runs independently of the underlying OS. The user accesses the app remotely without installing it.
- runs application software without installing it directly on the user's OS.
- This differs from complete desktop virtualization because only the application runs in a virtual environment—the OS on the end user's device runs as usual.



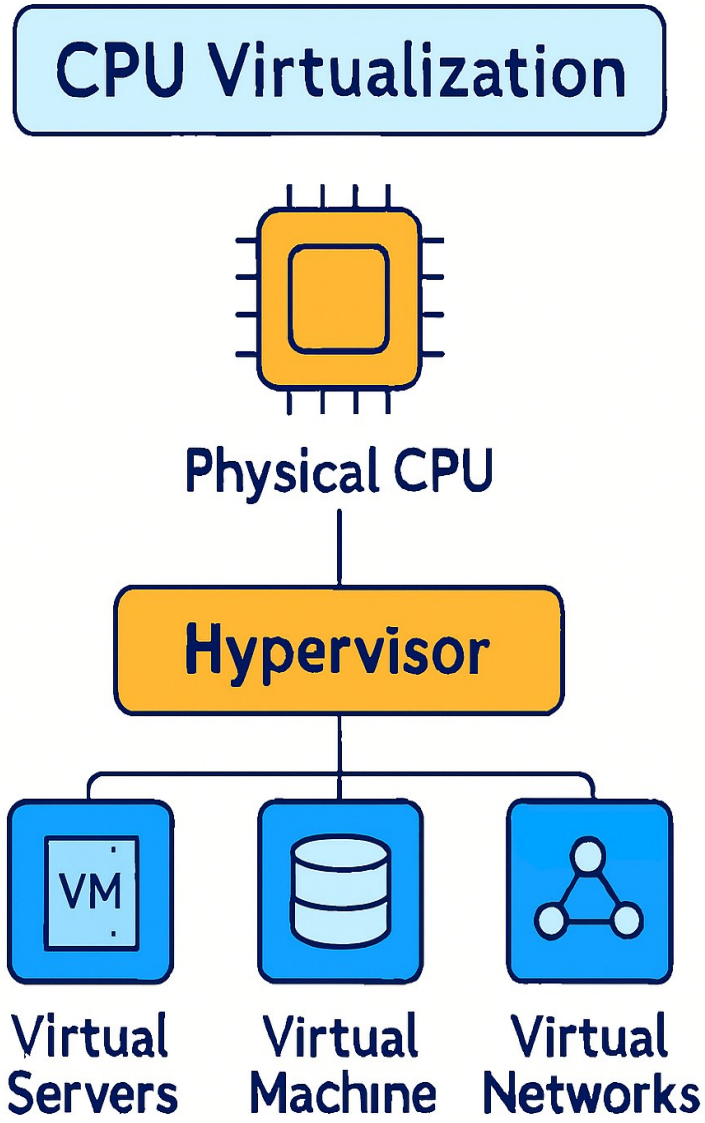
Data center Virtualization

- Abstracts most of a data center's hardware into software, effectively enabling an administrator to divide a single physical data center into multiple virtual data centers for different clients.
- Each client can access its own infrastructure as a service (IaaS), which would run on the same underlying physical hardware.



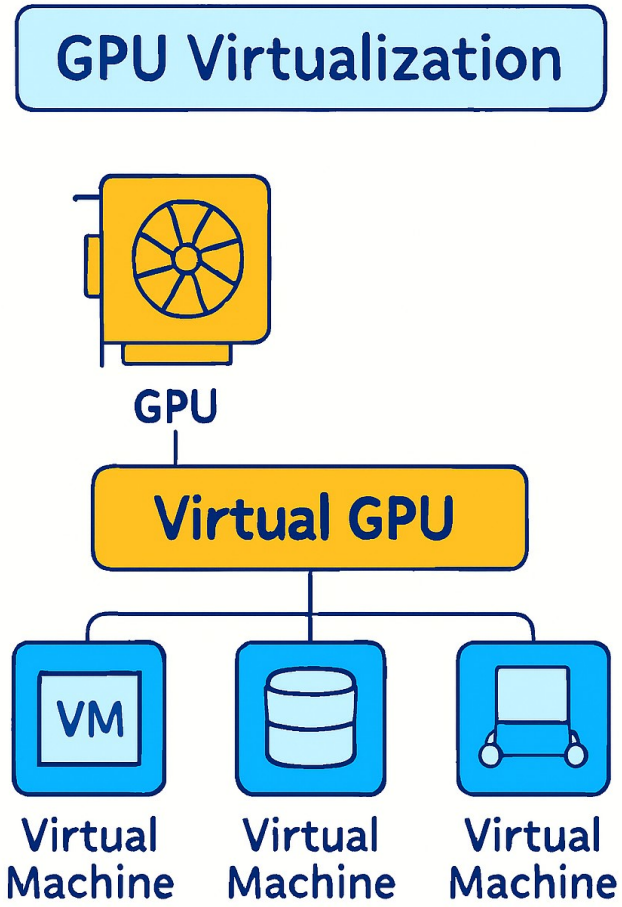
CPU Virtualization

- It allows a single CPU to be divided into multiple virtual CPUs for use by multiple VMs.
- At first, CPU virtualization was entirely software-defined, but many of today's processors include extended instruction sets that support CPU virtualization, which improves VM performance.



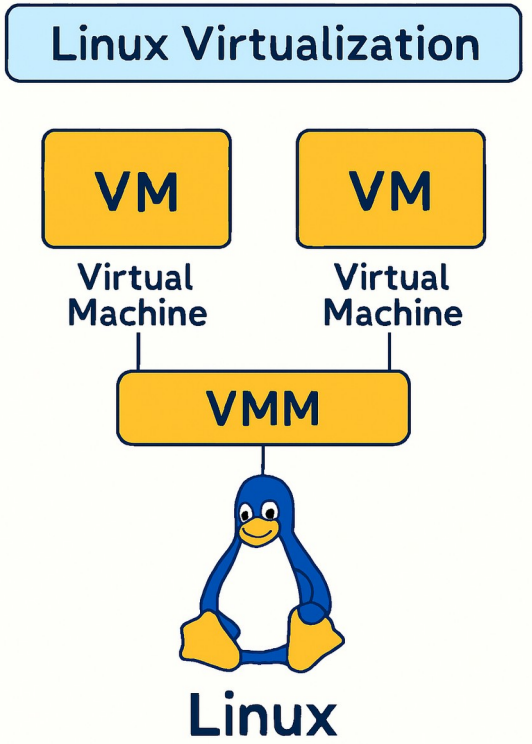
GPU Virtualization

- A GPU (graphical processing unit) is a special multi-core processor that improves overall computing performance by taking over heavy-duty graphic or mathematical processing.
- GPU virtualization lets multiple VMs use all or some of a single GPU's processing power for faster video, artificial intelligence (AI), and other graphic- or math-intensive applications.
- Pass-through GPUs make the entire GPU available to a single guest OS.
- Shared vGPUs divide physical GPU cores among several virtual GPUs (vGPUs) for use by server-based VMs.



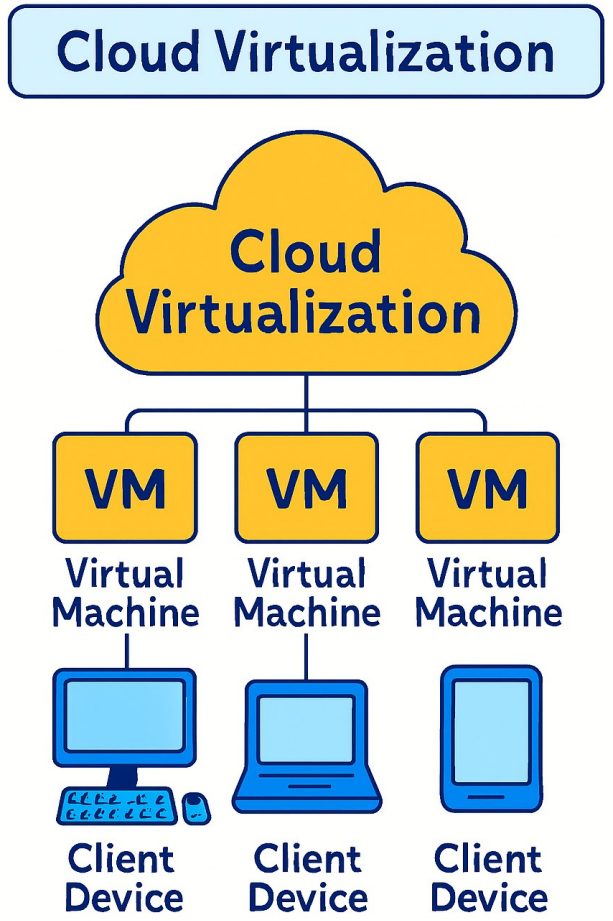
Linux Virtualization

- Linux includes its own hypervisor, called the kernel-based virtual machine (KVM), which supports Intel and AMD's virtualization processor extensions so you can create x86-based VMs from within a Linux host OS.
- As an open source OS, Linux is highly customizable.
- VMs can be created by running versions of Linux tailored for specific workloads or security-hardened versions for more sensitive applications.



Cloud Virtualization

- The cloud computing model depends on virtualization.
- By virtualizing servers, storage, and other physical data center resources, cloud computing providers can offer a range of services to customers, including the following:
- Infrastructure as a service (IaaS): Virtualized server, storage, and network resources can be configured based on their requirements.
- Platform as a service (PaaS): Virtualized development tools, databases, and other cloud-based services to build your own cloud-based applications and solutions.
- Software as a service (SaaS): Software applications used on the cloud. SaaS is the cloud-based service most abstracted from the hardware.



Docker installation on Multiple OS

- Docker images can support multiple platforms.
- When running an image with multi-platform support, docker automatically selects the image that matches your OS and architecture.
- Docker Official Images on Docker Hub provide a variety of architectures.
- For example, the busybox image supports amd64, arm32v5, arm32v6, arm32v7, arm64v8, i386. When running this image on an x86_64 / amd64 machine, the amd64 variant is pulled and run.

- BuildKit with Buildx is designed to work well for building images for multiple platforms and to run.
- When you invoke a build, set the --platform flag to specify the target platform for the build output, (for example, linux/amd64, linux/arm64, or darwin/amd64).
- When triggering a build, use the --platform flag to define the target platforms for the build output, such as linux/amd64 and linux/arm64:
- ***docker buildx build --platform linux/amd64,linux/arm64 .***

- Multi-platform images can be created using three different strategies that are supported by Buildx and Dockerfiles:

1) Using the QEMU emulation support in the kernel
2) Building on multiple native nodes using the same builder instance
3) Using a stage in Dockerfile to cross-compile to different architectures

1. QEMU

- QEMU is a hardware emulator that allows your system to simulate other CPU architectures (e.g., ARM64 on x86).
- Building multi-platform images under emulation with QEMU is the easiest way to get started if your builder already supports it.
- Using emulation requires no changes to your Dockerfile, and BuildKit automatically detects the architectures that are available for emulation.
- Docker Desktop supports running and building multi-platform images under emulation by default.

2. Multiple native nodes

- This strategy uses a builder instance that spans multiple physical or virtual machines, each with a different native architecture.
- You create a Buildx builder with multiple nodes:
- docker buildx create --name mybuilder --node node1 --node node2
- docker buildx use mybuilder
- Each node builds for its native platform, avoiding emulation overhead.

3. Multi-stage builds in Dockerfiles

- Multi-stage builds in Dockerfiles can be effectively used to build binaries for the platform specified with --platform using the native architecture of the build node.
- docker buildx build --platform linux/amd64,linux/arm64 .
- A list of build arguments like BUILDPLATFORM and TARGETPLATFORM is available automatically inside your Dockerfile and can be leveraged by the processes running as part of your build.

```
FROM --platform=$BUILDPLATFORM golang:alpine AS build

ARG TARGETPLATFORM

ARG BUILDPLATFORM

RUN echo "I am running on $BUILDPLATFORM, building for $TARGETPLATFORM" > /log

FROM alpine

COPY --from=build /log /log
```

- Run the ***docker buildx ls*** command to list the existing builders:

```
NAME/NODE        DRIVER/ENDPOINT      STATUS          PLATFORMS
default                docker               running         linux/amd64
mybuilder *      docker-container     running         linux/amd64, linux/arm64, linux/arm/v7
```

- The default builder usually only supports your host architecture.
- A docker-container builder (created with docker buildx create) can support multiple platforms, especially when combined with QEMU emulation or multiple native nodes.
- This command helps you confirm whether your setup is ready for multi-platform builds.

THANK YOU